```python
from sqlalchemy import create_engine, Column, Integer, String, ForeignKey
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker, relationship, joinedload
import logging


logging.basicConfig()
logging.getLogger('sqlalchemy.engine').setLevel(logging.INFO)


engine = create_engine('sqlite:///blog.db', echo=True)  # echo=True включает вывод
SQL
Base = declarative_base()

class User(Base):
    __tablename__ = 'users'
    id = Column(Integer, primary_key=True)
    name = Column(String(50), nullable=False)
    email = Column(String(100), unique=True)


    posts = relationship("Post", back_populates="author", cascade="all, delete-
orphan")

class Post(Base):
    __tablename__ = 'posts'
    id = Column(Integer, primary_key=True)
    title = Column(String(200))
    content = Column(String)
    user_id = Column(Integer, ForeignKey('users.id'))

    author = relationship("User", back_populates="posts")


Base.metadata.create_all(engine)
Session = sessionmaker(bind=engine)
session = Session()
```

Часть 1. CRUD-операции

```python
users_data = [
```

```
    User(name="Анна Смирнова", email="anna@example.com"),
    User(name="Иван Петров", email="ivan@example.com"),
    User(name="Мария Кузнецова", email="maria@example.com")
]
session.add_all(users_data)
session.commit()



posts = [
    Post(title="Первый пост Анны", content="Привет, мир!", author=users_data[0]),
    Post(title="Второй пост Анны", content="Ещё одна запись",
author=users_data[0]),
    Post(title="Пост Ивана о Python", content="Учим ORM", author=users_data[1]),
    Post(title="Иван о SQL", content="Запросы под капотом", author=users_data[1]),
    Post(title="Мария о путешествиях", content="Еду в горы",
author=users_data[2]),
    Post(title="Мария о фото", content="Новый объектив", author=users_data[2])
]
session.add_all(posts)
session.commit()



user = session.get(User, 1)
print(f"Найден пользователь: {user.name}")



user.name = "Анна Ивановна Смирнова"
session.commit()



user_to_delete = session.get(User, 2)
session.delete(user_to_delete)
session.commit()
```

Часть 2. Работа со связями

```
users = session.query(User).all()

for user in users:
```

```
    print(f"\nПользователь: {user.name}")
    for post in user.posts:
        print(f"  - Пост: {post.title}")
```

Часть 3. Анализ работы ORM

Вопрос 1: Сколько SQL-запросов выполняется?

```
SELECT users.id, users.name, users.email FROM users
SELECT posts.id, posts.title, posts.content, posts.user_id FROM posts WHERE
posts.user_id = ?
SELECT posts.id, posts.title, posts.content, posts.user_id FROM posts WHERE
posts.user_id = ?
SELECT posts.id, posts.title, posts.content, posts.user_id FROM posts WHERE
posts.user_id = ?
```

Ответ: выполняется **4 SQL-запроса**:

- 1 запрос на получение всех пользователей
- 3 запроса на получение постов (по одному на каждого пользователя)

Вопрос 2: Почему выполняется именно столько?

Потому что ORM использует **ленивую загрузку (lazy loading)**. Когда мы обращаемся к `user.posts`, ORM сначала проверяет, загружены ли уже посты. Если нет — она выполняет отдельный SQL-запрос для этого пользователя.

Часть 4. Проблема N+1

Определение: Да, возникает проблема N+1.

Суть проблемы:

```
N+1 = 1 запрос на получение N пользователей + N запросов на получение постов для
каждого пользователя
```

В нашем случае: N = 3 пользователя → 1 + 3 = 4 запроса.

Почему это плохо?

- Если пользователей 1000 → выполнится 1001 запрос
- Каждый запрос — это сетевой вызов, парсинг, выполнение на БД
- Резкое падение производительности при масштабировании

Часть 5. Оптимизация (Eager Loading)

```
from sqlalchemy.orm import joinedload



users = session.query(User).options(joinedload(User.posts)).all()

for user in users:
    print(f"\nПользователь: {user.name}")
    for post in user.posts:
        print(f"  - Пост: {post.title}")
```

Сравнение количества запросов:

Вариант	Количество SQL-запросов
Без оптимизации (ленивая загрузка)	N + 1 (4 для 3 пользователей)
С оптимизацией (eager loading)	1 или 2 (в зависимости от типа JOIN)

При использовании `joinedload` выполнится **1 запрос** с LEFT OUTER JOIN:

```
sql

SELECT users.id, users.name, users.email, posts.id, posts.title, posts.content,
posts.user_id
FROM users LEFT OUTER JOIN posts ON users.id = posts.user_id
```

Альтернатива: `selectinload` — выполнит 2 запроса, но эффективнее для больших объёмов.

Контрольные вопросы (письменные ответы)

1. Что такое ORM?

ORM (Object-Relational Mapping) — технология, позволяющая работать с реляционной базой данных через объектно-ориентированный код, без написания SQL-запросов вручную. Таблицы превращаются в классы, строки — в объекты, а связи — в ссылки между объектами.

2. Какие преимущества он даёт?

- **Ускорение разработки** — не нужно писать шаблонный SQL
- **Безопасность** — автоматическая защита от SQL-инъекций
- **Переносимость** — смена БД (PostgreSQL → MySQL) требует минимальных правок
- **Удобство работы со связями** — обращение `user.posts` вместо ручных JOIN-ов
- **Поддержка миграций** — автоматическое обновление схемы БД

3. В чём заключается проблема N+1?

Проблема возникает при выборке коллекции объектов с последующим обращением к связанным данным. ORM по умолчанию выполняет:

- 1 запрос для получения родительских объектов
- N дополнительных запросов (по одному на каждый объект) для подгрузки связанных данных

Пример: выборка 100 пользователей и вывод их постов → 101 запрос вместо 1-2 оптимальных.

4. Когда ORM использовать не стоит?

- **Сложные аналитические отчёты** (много JOIN, GROUP BY, оконных функций)
- **Высоконагруженные системы** (каждый лишний запрос критичен)
- **Массовые операции** (UPDATE/INSERT тысяч записей — быстрее сырым SQL)
- **Специфические фишки БД** (JSONB-операторы PostgreSQL, полнотекстовый поиск)
- **Обучение SQL** (новичкам полезно сначала понять сам SQL)

Дополнительное задание (пример)

Реализация с JOIN вручную (чистый SQL через ORM)

```
from sqlalchemy.orm import Session

results = session.query(User, Post).join(Post, User.id == Post.user_id).all()

for user, post in results:
    print(f"{user.name} написал: {post.title}")


import time


start = time.time()
users = session.query(User).all()
for u in users:
    _ = u.posts
print(f"N+1: {time.time() - start:.4f} сек")
```

```python
start = time.time()
users = session.query(User).options(joinedload(User.posts)).all()
for u in users:
    _ = u.posts
print(f"Eager: {time.time() - start:.4f} сек")



start = time.time()
connection = engine.connect()
result = connection.execute("""
    SELECT users.name, posts.title
    FROM users LEFT JOIN posts ON users.id = posts.user_id
""")
rows = result.fetchall()
print(f"Raw SQL: {time.time() - start:.4f} сек")
```